

You started a task

Find issues, bugs, bad practises, old frameworks or api calls, inefficient code usage, anti patterns and other things that may be overengineered, too complex or might be risky and have high chance of failure.

Also, look at other best practises I can apply. Find the best practises and modern standards for the frameworks I use.

I also want some advice and things to do, and not to do, when packaging and releasing. Should it be self contained? Should users have the dependencies? How to optimize this more? App is very heavy and prone to failure, I think.

Maybe it's worth checking out other WinUI3 projects and searching for best practises.

I saw Windows App SDK V2 also brought some new API calls an app can use. Maybe we can migrate or modernize some more. The less code, the better. The more modern, the better.

The better the code is commented/documented, the better.

Keep in mind, that We don't want to lose functionality.

Refactoring code and exploring best practices for WinUI3

Plan dismissed

Audit summary (no code changes made):

- **Broken repo ownership/URLs (high impact bug):** many places still point to `REALSDEALS/pchealth`, which can break update checks and send users to wrong pages.
 - `src/gui/pchealth/services/updateschecker.cs:122`
 - `src/gui/pchealth/winuiwindow.xaml.cs:62`
 - `src/gui/pchealth/pages/info/page.xaml:45-47`
 - `src/CLI/Menu/Main.ps1:49-54`
 - plus README/SECURITY/docs links.
- **Release dependency mismatch (high risk):** build/release doc still says Windows App SDK 1.8, project uses 2.0.1.
 - `development/tools/Build-Release.ps1:9-18` vs `src/gui/pchealth/pchealth.csproj:49`
- **Missing launcher reference bug:** repo references `src/CLI/start.sh`, but file does not exist.
 - `README.md:53`, `src/gui/start.ps1:14`, `github/labeler.yml:104`
- **Heavy/fragile code hotspot:** `HealthPage.xaml.cs` is very large and does too much (collection, parsing, scoring, UI rendering), increasing failure risk and maintenance cost.
- **Inefficient file reading:** `CrashPage` reads the full CBS log into memory just to show last 200 lines.
 - `src/gui/pchealth/pages/crash/page.xaml.cs:32-41`
- **Reliability issue on startup:** update check uses `HttpClient.GetAsync` without timeout/cancellation; can feel like random startup lag/failure.
 - `updateschecker.cs`
- **Error handling anti-patterns:** several broad/empty catches suppress root causes (example: logger and multiple pages/scripts).

Best-practice direction (WinUI 3 + .NET + PowerShell):

1. **Stability first:** keep APIs modern (you already use CIM and avoid `log` / `MI` in C#), but replace broad catches with specific exceptions + structured logging.
2. **Reduce complexity:** split `HealthPage` logic into small services (hardware, storage, security, battery), keep page as orchestration/UI only.
3. **Unified process execution:** use one shared runner pattern everywhere with `timeout` + `cancellation` + `explicit exit-code handling`.
4. **Documentation correctness over volume:** prioritize accurate dependency/release docs before adding more comments.

Packaging/release advice (what to do / not do):

Do

- Ship two artifacts:
 - **Default user build:** self-contained (maximum reliability, fewer support issues).
 - **Advanced/smaller build:** framework-dependent (smaller download, requires runtimes).
- Add a reproducible release workflow (build, sign, hash, smoke test on clean Windows VM).
- Keep SHA256 and publish exact runtime requirements.
- Test install/run on a machine without SDK/toolchain.

Don't

- Don't require end users to have `.NET SDK` for normal usage (runtime only for framework-dependent build).
- Don't aggressively enable trimming/AOT for WinUI 3 without deep validation (high regression risk due reflection/WinRT).
- Don't silently swallow exceptions in critical flows (startup, update, `pr` launch, data gather).

Should it be self-contained?

- For **normal users:** yes, prefer self-contained (or Windows App SDK bundled strategy) for reliability.
- For **size-sensitive users:** offer framework-dependent as secondary download.
- **Best balance:** publish both and make the reliable one the default.

Highest-value next steps (safe, no functionality loss):

1. Fix all repo URL ownership references (`REALSDEALS` → `Stensels` where intended).
2. Align release docs/scripts with actual Windows App SDK/.NET runtime requirements.
3. Fix `start.sh` references or add the file if intended.
4. Refactor `HealthPage` incrementally into services (no behavior changes).
5. Optimize CBS log tail reading and add timeout/cancellation to update checks.

You asked

Thanks. Can you find more? More improvements and suggestions? I will listen. However, the repo owner will stay on REALSDEALS because this is a WIP that will eventually get merged back into their upstream branch.

Discussing additional improvements and suggestions

Plan dismissed

Got it — I'll keep the GitHub owner as REALSDEALS in all recommendations for this WIP/upstream flow.

More improvements I found:

- **Boot/Repair confirmation mismatch (bug):** UI says "Type CONFIRM to proceed" but there is no input field, only Proceed/Cancel (`/home/runner/work/pchealth/pchealth/src/gui/pchealth/pages/bootrepair/page.xaml.cs`).
- **Ninite supply-chain risk:** downloads and executes EXE directly with no hash/signature verification (`.../NinitePage.xaml.cs`).
- **Ninite memory inefficiency:** `GetBytesToArrayAsync` loads full binary into memory; use streamed download for reliability on slower systems (`.../NinitePage.xaml.cs`).
- **Global admin requirement risk:** app manifest forces `requireAdministrator` for entire GUI; increases failure surface and security blast radius (`.../src/gui/pchealth/app.manifest`).
- **Logger reliability gap:** silent catches + no rotation/max-size means silent failure and potentially huge log file (`.../Services/Logger.cs`).
- **Health page complexity hotspot:** one very large file with collection (`log/scoring/UI` all together (`.../Pages/HealthPage.xaml.cs`)).
- **Health page sync-over-async pattern:** repeated `GetAwaiter().GetResult()` and blocking process reads; fragile over time (`.../HealthPage.xaml.cs`).
- **DISM feature checks can partially vanish:** `Task.WaitAll(..., task)` then only completed tasks are used; incomplete checks are dropped silently (`.../HealthPage.xaml.cs`).
- **CBS log page reads whole file for tail:** heavy memory/time on large logs (`.../Pages/CBSLogPage.xaml.cs`).
- **Network pages can degrade over long runs:** repeated `outputText.Text +=` string concatenation is expensive (`.../NetworkContinousPage.xaml.cs`, `.../TraceroutePage.xaml.cs`).
- **Programs catalog mutability risk:** static `programs` objects are mutated (`IsInstalled`) and reused globally; can cause stale/shared state issues (`.../Pages/ProgramsPage.xaml.cs`).
- **Release builder quality gap:** release tool uses `dotnet build` + copy instead of `dotnet publish`; less deterministic packaging (`.../development/tools/Build-Release.ps1`).
- **Release prerequisites mismatch:** script comments still reference Windows App SDK 1.8 while project uses 2.0.1 (`.../Build-Release.ps1`, `.../pchealth.csproj`).
- **No signed release pipeline:** no artifact signing/attestation/SBOM generation in workflow (all workflows under `.../github/workflows`).
- **No clean-machine smoke validation for release artifacts:** release process builds/tips but doesn't validate install/run path on fresh environment.
- **Security policy link consistency:** SECURITY advisory URL and docs should stay aligned with REALSDEALS upstream policy (`.../SECURITY.md`).

Best-practice suggestions (modern + low-risk first):

1. **Reliability first:** add consistent timeout/cancellation/retry policy for all network/process operations.
2. **Least privilege path:** keep admin where required, but consider splitting elevated operations into helper process instead of full-app elevation.
3. **Refactor HealthPage** by domain services (CPU/GPU/storage/security/...etry) without changing behavior.
4. **Use publish for releases** (RID-specific), then package only published `.../out`.
5. **Add release hardening:** signing, SBOM, provenance/attestation, and post-build smoke tests.
6. **Add bounded UI logs/output buffers** to prevent long-session slowdown.
7. **Strengthen download trust model** for external binaries (checksum/signature validation).
8. **Improve observability:** structured logging + rotation + explicit user-safe diagnostics.
9. **Keep docs synced with runtime reality** (dependencies, launcher paths, support matrix).
10. **For Windows App SDK 2 modernization:** target replacing old compatibility workarounds and centralize title bar/windowing/theme behavior in one service to reduce page-level code.

If you want, I can next give you a prioritized "top 10 fixes by impact vs effort" list tailored for a first stabilization pass.

You asked

Thanks for the suggestions, now find more. Maybe look at some apps that BuiltByBel has made (Belim). We could possibly learn a lot from his best practises. Good suggestions so far, I want more. I am making a todo list for optimization of my own apps.

Exploring best practices for app optimization

Plan dismissed

Great direction. I kept the upstream owner assumption as REALSDEALS and found more items for your optimization todo list:

- **Fix launcher inconsistency:** `/home/runner/work/pchealth/pchealth/src/gui/pchealth/CLI/start.ps1:14` still tells users to run `src/CLI/start.sh` (doesn't exist).
- **Runs docs mismatch:** `/home/runner/work/pchealth/pchealth/README.md:131` says Boot Repair includes CHKDSK+SFC+BOOTREC, but GUI boot repair runs BOOTREC commands only.
- **Fix UX copy bug:** BootRepair says "Type CONFIRM to proceed" but has no textbox (`.../pages/bootrepair/page.xaml.cs`).

High-impact optimization ideas (BuiltByBel-style "safe + clean UX"):

1. Add a "dry run / preview mode" for destructive tools (network reset, boot repair, scan/repair).
2. Add before/after snapshots (network config, services state, boot entries) so users can export diagnostics.
3. Add rollback helpers where possible (e.g. restore network adapter settings).
4. Add one-click "copy diagnostics" bundle (version, OS, logs, last command outputs).
5. Add bounded output buffers in long-running pages to prevent UI slowdown.
6. Switch heavy text append paths to `StringBuilder/BufferedLogView` instead of repeated `Text.AppendLine`.
7. Add consistent command timeout policy per tool class (short/medium/long operations).
8. Add standard retry/backoff for network calls (update checks/downloads).
9. Harden downloader flows (Ninite): stream download + checksum/signature validation + size sanity checks.
10. Add explicit trust prompts before launching downloaded binaries.
11. Reduce global elevation risk by moving privileged actions to a small elevated helper instead of full-app admin manifest.
12. Add log rotation + max size to logger (`.../Services/Logger.cs`) and stop silent swallow.
13. Add structured event IDs for logs to make support triage faster.
14. Split large `HealthPage.xaml.cs` into domain services (storage/cpu/security/network) for maintainability and faster iteration.
15. Remove sync-over-async hotspots in health checks (`GetAwaiter().GetResult()` usage) to reduce deadlock/latency risk.
16. For DISM feature checks, show "timed out" state instead of silently dropping incomplete tasks.
17. Move release packaging from `dotnet build` + copy to `dotnet publish` artifact-first packaging.
18. Align release docs with current dependencies (Build-Release still mentions WinAppSDK 1.8).
19. Add release pipeline steps: SBOM + artifact attestations/provenance + optional signing.
20. Add smoke-test job that starts built CLI/GUI artifacts in clean runners.
21. Bring change-log/workflow reality back in sync (change/refs deprecated-patterns workflow that isn't present in `github/workflows`).
22. Add imports/export/reset (Belim-style power-user convenience) for reproducible setups.
23. Add feature flags for risky/experimental tools (so WIP features can ship disabled by default).
24. Add health score trend/history (local cache) so users see changes over time, not just one snapshot.
25. Add non-admin read-only mode for diagnostics-only paths to improve accessibility and safety.

If you want, I can next produce a prioritized Top-15 (impact vs effort) list with "quick wins this week" vs "deeper refactors."

You asked

But, like I asked: see what we can learn from <https://github.com/BuiltByBel>

Analyzing BuiltByBel repository for insights

Plan dismissed

Got it — I checked BuiltByBel repos (PyOOBE, Bloaynslo, CrapFixer, TidyOS, ThisIsWin11, privatezilla). Here are more concrete best-practice ideas you can copy into your optimization todo list:

1. **Plugin-first architecture** (JSON/PS1 plugins + manifest) instead of hardcoding all tweaks.
2. **Check / Do / Undo model** per tweak (state detection, apply, rollback).
3. **Per-tweak safety metadata** (requires admin, reboot needed, risk level, reversible yes/no).
4. "Create restore point" one-click preflight before running risky operations.
5. "Analyze first, fix second" flow with clear status colors (already optimized vs actionable).
6. Human-readable tweak descriptions and in-app help per item.
7. Remote plugin manifest update channel so new tweaks ship without full app release.
8. Community plugin ecosystem (curated, optional, signed/verified sources).
9. Data-driven bloat lists (external feed files like app lists/filters).
10. Preset filters (gaming, privacy, office, low-end PC, minimal install).
11. Operation logs + web/local log analyzer for support and troubleshooting.
12. Portable executable mode (no install required) for technician workflows.
13. Just-in-time elevation (default non-admin app, elevate only on privileged actions).
14. Strong rollback UX ("undo available" badge and dedicated restore actions).
15. Batch execution queue with progress, per-step output, and partial-failure handling.
16. Plugin categories/tags (privacy, UI, performance, cleanup, networking).
17. Safe defaults enabled — advanced/risky tweaks hidden behind explicit opt-in.
18. Plain-language user messaging (what changes, why, and how to revert).
19. Flat update checks via manifest/release API and simple "download latest" UX.
20. Feature consolidation strategy (merge old tools into one core app with modular packs).

If you want, I can now turn this into a prioritized backlog for pchealth.

Quick wins (1-2 days), Medium (1 week), and Big bets (multi-week).

You asked

Look more on it's newer project and repo's. Since I need things WinUI3 related to compare. Think of a FluentCleaner or a Winslopr. Maybe I can learn from those apps and how they are built as well. Thank you.

Researching FluentCleaner and Winslopr applications

Plan dismissed

What pchealth's WinUI 3 GUI can learn from FluentCleaner & Winslopr

Both projects are recent, production-quality WinUI 3 apps by the same author and share a tight design vocabulary that maps directly onto pchealth's GUI. Here are concrete, actionable lessons organised by area:

1 — Project / build setup

Both use	pchealth status	Action
<code>net10.0-windows10.0.19041.0</code> as TargetFramework	pchealth uses <code>net9.0</code> (verify in <code>csproj</code>)	Consider targeting <code>net10.0</code> when .NET 10 ships stable
<code>WindowsPackageType=None</code> (unpackaged, self-contained EXE)	verify <code>CopyToOutputDirectory=PreserveState</code>	Keep unpackaged for portable/no-UAC use cases
<code>SelfContained=true</code> + separate <code>win-x64</code> / <code>win-arm64</code> publish	probably not done	Needed for proper standalone distribution
<code>Microsoft.WindowsAppSDK 1.8</code> (FluentCleaner's pin)	pchealth uses 2.0.1 — newer	pchealth is already ahead here — don't downgrade
<code>CommunityToolkit.Mvvm</code> (FluentCleaner)	not present in pchealth	Add for MVVM bindings (ObservableObject, RelayCommand)
<code>AssemblyVersion</code> set to a <code>YY.MM.BUILD</code> calendar scheme (e.g. 26.05.02)	pchealth reads from repo-root <code>version</code> file	Both approaches are fine, ensure single source of truth

2 — App.xaml.cs bootstrap patterns (copy/adapt these)

Theme application (both projects have this; pchealth should too):

```
using Microsoft.UI.Xaml;

// Apply to the window root + sync titlebar buttons
public static void ApplyTheme(Window window, string? theme)
{
    if (window.Content is FrameworkElement root)
    {
        root.RequestedTheme = theme switch { "Dark" => ElementTheme.Dark, "Light" => ElementTheme.Light, _ => ElementTheme.Default };
    }
    // also set TitleBar.ButtonForegroundColor so caption buttons match
}
```

pchealth currently has no theme switching; adding `ApplyTheme()` in `App.xaml.cs` is a self-contained change.

Window size restore on launch / save on close (FluentCleaner):

Saves `Width/Height` to settings on `window.Closing`, restores + centres on next launch. Add the same `RestoreWindowSize()` + `CenterWindow()` pair to pchealth's `App.xaml.cs`.

Transparent title bar button background (FluentCleaner's `SetupTitleBar()`):

```
using Microsoft.UI.Xaml;

bar.ButtonBackground = Colors.Transparent;
bar.ButtonContentBackground = Colors.Transparent;
```

Only override idle state; let WinUI handle hover/pressed — overriding all states breaks hover.

Crash log on unhandled exception (Winslopr):

```
using System;

this.UnhandledException += (s, e) => {
    File.AppendAllText("crash.log", $"{DateTime.Now} {e.Exception}\n\n");
    e.Handled = false;
};
```

A one-liner that gives users a file to attach in bug reports.

Parallel feature preload on launch (Winslopr):

```
using System;

// Start loading in the background while the window is being created
PreloadTask = Task.Run(() => LoadFeatures());
// ... then await PreloadTask in the first page's OnNavigatedTo
```

Eliminates the "white flash" delay when opening the first page of content.

Compact density mode toggle (Winslopr):

```
using System;

Application.Current.Resources.TryGetValue("Density", out ResourceDictionary? source);
new ResourceDictionary { Source = new Uri("ms-appx:///Microsoft.UI.Xaml/DensityStyles/Compact.xaml") };
```

Free performance/space win for power users, applied once at startup.

3 — AppInfo.cs pattern (both projects, identical approach)

A static `AppInfo` class that reads the assembly version and exposes:

- `VersionString` → formatted `YY.MM.BUILD`
- `DisplayVersion` → adds `(Insider)` suffix when `IsInsider = true`
- `InsiderSubtitle` → empty string or `Insider` bound to `FluentCleaner.SubTitle`

pchealth reads version from the repo-root `version` file — this is fine but slightly more complex. If switching to assembly version, the `AppInfo.cs` pattern is clean and zero-dependency.

4 — MainWindow.xaml patterns

FluentCleaner uses the new WinUI 3 `<TitleBar>` control (requires WindowsAppSDK 1.6+):

```
<TitleBar x:Name="AppTitleBar"
          Title="FluentCleaner"
          SubTitle="FluentCleaner SubTitle">
  <TitleBar.Content> <!-- global search in the drag region -->
  <TitleBar.RightAdorners> <!-- [...] new button -->
```

And in code-behind:

```
using Microsoft.UI.Xaml;

extendsContentFromAppTitleBar = true;
setTitleBar(AppTitleBar);
AppWindow.TitleBar.PreferredDarkThemeColor = TitleBarThemeOption.Default;
```

pchealth likely uses the older `SetTitleBar()` + custom `Grid` approach. Migrating to `<TitleBar>` is low-risk and gives native drag handling, icon, subtitle, and built-in content slots for free.

The responsive search collapse trick (FluentCleaner):

Between 560–700 px collapse search `TextBox`, show icon `Button` + flyout. Between 700–760 px shrink to 220 px. Above 760 px full 260 px.

Driven by `SizeChanged` event — no XAML state manager needed.

Winslopr uses a bottom nav pill instead of `NavigationView`:

A custom `TaskView` of `Button` inside a `Border` with `CornerRadius="22"`. This is an intentional design choice for a compact single-window app. pchealth's `NavigationView` is more conventional and fine as-is.

The `DrilldownNavigationTransitionInfo` for page transitions (FluentCleaner):

```
using Microsoft.UI.Xaml;

ContentFrame.Navigate(page, null, new DrilldownNavigationTransitionInfo());
```

One-line change that adds a zoom-in feel on navigation — much better than the default fade.

5 — Page interface contracts

FluentCleaner defines two thin interfaces that the shell and pages share:

```
using System;

interface ISearchChallenge { void OnSearch(string text); }
interface IPageActivation { void BuildActionMenu(out Flyout? flyout); }
```

Winslopr uses a similar pattern:

```
using System;

interface IMainActions { Task AnalyzeAsync(); Task FixAsync(); }
interface ISearchable { void ApplySearch(string query); }
interface IView { void RefreshView(); }
```

pchealth should adopt a similar pattern — each `Page` class opts into capabilities by implementing the relevant interface. The shell (`MainWindow`) then checks `ContentFrame.Content is ISearchChallenge` etc. This replaces ad-hoc casting.

6 — Feature/Tweak model (Winslopr's FeatureBase — the most important pattern)

This is the architecture you should study most closely for pchealth's optimization page:

```
using System;

public abstract class FeatureBase {
    public abstract string ID(); // display name
    public abstract string GetFeatureDetails(); // tooltip / detail text
    public abstract Task<bool> CheckFeature(); // read current state (sync)
    public abstract Task<bool> ToggleFeature(); // apply change (async)
    public abstract bool UnderFeature(); // revert change
    public virtual bool IsApplicable(); // no version guard
    public virtual string GetFeatureReason(); // user-facing explanation
}
```

Each concrete feature (e.g. `DisableWindowsUpdate`, `RemoveCortana`) is a 30–60 line class with no UI code. The UI only calls `CheckFeature()` / `ToggleFeature()` / `UnderFeature()`.

pchealth currently uses a page-per-feature pattern with logic mixed into `*.xaml.cs`. Introducing a `FeatureBase`-style abstraction would:

- Make it trivial to add new tweaks without touching the UI
- Enable a proper "Analyze all" → show what needs fixing → Apply selected" flow
- Enable per-feature undo without storing state in the page

7 — FeatureTreeItem (Winslopr) — tree node as ViewModel

A self-contained `IMultiValuePropertyChanger` class that acts as both the data model and the tree node:

- `IsChecked` cascade: checking a category checks all children
- `AnalysisStatus` enum → drives text/dot color via `StatusIconConverter` (Green=OK, Red=NeedsFix, Orange=NotApplicable)
- `FilteredChildren` rebuilt on search (never toggle visibility on `TreeViewItem` — WinUI 3 crashes)
- `IsPinned` flag (has a `ScriptPath`) keeps plugin nodes and feature nodes in one tree

8 — Settings storage

FluentCleaner	Winslopr
<code>AppGlobalAppInfo.Settings.Json</code> via <code>System.Text.Json</code>	<code>AppInfo</code> Winslopr.txt with <code>key=value</code> lines
<code>AppSettings.Instance</code> singleton, <code>Reload()</code> on startup	<code>SettingsManager.Get/Set/Has/Is/Setting</code> static helpers
Typed properties with defaults, saved on <code>Window.Closing</code>	JSON-based, plain-text, no schema

Recommendation for pchealth: Use the FluentCleaner approach (`ISettings` in `AppGlobal`) — it's already typed, survives path changes, and supports complex objects like `HashSet<string>` / `HashSet<int>`.

9 — Logger (Winslopr — Logger.cs)

A framework-agnostic static logger with:

- `LogLevel` enum (Info / Warning / Error / Custom)
- static event `Action<LogEntry>` `LogAdded` → any view subscribes without knowing about others
- `BeginSection(name)` → prints `===== SECTION (Name:cs) =====`
- `EndOnlyList<LogEntry>` entries for copy/export

pchealth's GUI currently has no operation log visible to the user. Wiring a `RichTextBlock` to `Logger.LogAdded` (as Winslopr does) gives users a live activity feed without any architectural cost.

10 — Localization (Winslopr)

Winslopr uses `Localize` on every UI string + `Strings/en/resources.resx`, `Strings/de/resources.resx`, `Strings/fr/resources.resx`. The `Localize.cs` helper resolves strings by key with a fallback.

This is an intentional design choice for a compact single-window app. pchealth has no localization currently — adding `Localize` to XAML is a prerequisite that should be done before the UI grows further, even if only English ships initially.

11 — Misc small wins

- `MinimizeBox` vs `MaximizeBox` kinds="basealt" — BaseAlt (Winslopr) is darker, more distinct. Try both.
- Acrylic fallback for settings overlays — Winslopr's settings panel uses `AcrylicBrush` with `FallbackColor` for when OS transparency is disabled. Copy this pattern for any overlay/flyout.
- `TeachingTip` for first-run onboarding — Winslopr shows a `TeachingTip` once to introduce the Plugins feature. Dismissed state is stored in settings.

- `UseDefaultAppSettings` for grouped actions — Winslopr uses `OpenFileDialog` for the "Actions" group (Toggle All / Undo / Refresh / Log...). Cleaner than a separate toolbar.
- `SettingsManager.HasFlag / SetFlag` — boolean app state (e.g. "shown onboarding") stored as `HasKeyTrue` lines in a plain text file. Simpler than registry for non-critical state.

- `OnBeforePerfCheck` — Winslopr has a dedicated helper for OS version detection (`IsWindows11`, `BuildNumber`, etc.) rather than scattering `Environment.OSVersion` checks. Worth extracting the same in pchealth.